```
-- ============================================
-- KIRANA SHOP MANAGEMENT SYSTEM - COMPLETE DATABASE SCHEMA
-- ============================================
-- Version: 1.0
-- Database: PostgreSQL
-- Multi-tenant: Yes (retailer_id in all tables)
-- ============================================

-- ============================================
-- 1. RETAILERS TABLE
-- ============================================
CREATE TABLE retailers (
    id BIGSERIAL PRIMARY KEY,
    business_name VARCHAR(255) NOT NULL,
    owner_name VARCHAR(255) NOT NULL,
    email VARCHAR(255) UNIQUE,
    phone_number VARCHAR(15) NOT NULL,
    address TEXT,
    gst_number VARCHAR(15),
    business_type VARCHAR(50),
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    deleted_at TIMESTAMP NULL
);

CREATE INDEX idx_retailers_active ON retailers(is_active) WHERE deleted_at IS NULL;

COMMENT ON TABLE retailers IS 'Master table for all kirana shops using the system';
COMMENT ON COLUMN retailers.business_type IS 'kirana, general_store, medical, etc.';

-- ============================================
-- 2. SUPPLIERS TABLE
-- ============================================
CREATE TABLE suppliers (
    id BIGSERIAL PRIMARY KEY,
    retailer_id BIGINT NOT NULL,
    company_name VARCHAR(255) NOT NULL,
    contact_person VARCHAR(255),
    phone VARCHAR(15),
    email VARCHAR(255),
    address TEXT,
    gst_number VARCHAR(15),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    deleted_at TIMESTAMP NULL,
    FOREIGN KEY (retailer_id) REFERENCES retailers(id) ON DELETE CASCADE
);

CREATE INDEX idx_suppliers_retailer ON suppliers(retailer_id) WHERE deleted_at IS NULL;

COMMENT ON TABLE suppliers IS 'Wholesalers/distributors from whom retailers purchase inventory';

-- ============================================
-- 3. CUSTOMERS TABLE
-- ============================================
CREATE TABLE customers (
    id BIGSERIAL PRIMARY KEY,
    retailer_id BIGINT NOT NULL,
    name VARCHAR(255) NOT NULL,
    phone_number VARCHAR(15) NOT NULL,
    email VARCHAR(255),
    address TEXT,
    total_purchases DECIMAL(12,2) DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    deleted_at TIMESTAMP NULL,
    FOREIGN KEY (retailer_id) REFERENCES retailers(id) ON DELETE CASCADE
);

CREATE INDEX idx_customers_retailer ON customers(retailer_id) WHERE deleted_at IS NULL;
CREATE INDEX idx_customers_phone ON customers(retailer_id, phone_number);
```

```
COMMENT ON TABLE customers IS 'Customer information for credit sales and purchase tracking';

-- ============================================
-- 4. ITEMS TABLE
-- ============================================
CREATE TABLE items (
    id BIGSERIAL PRIMARY KEY,
    retailer_id BIGINT NOT NULL,
    name VARCHAR(255) NOT NULL,
    description TEXT,
    sku VARCHAR(100),
    barcode VARCHAR(100),
    category VARCHAR(100),
    unit VARCHAR(50),
    purchase_price DECIMAL(10,2) NOT NULL,
    selling_price DECIMAL(10,2) NOT NULL,
    current_stock DECIMAL(10,2) DEFAULT 0,
    min_stock_threshold DECIMAL(10,2) DEFAULT 0,
    reorder_point DECIMAL(10,2) DEFAULT 0,
    max_stock_level DECIMAL(10,2),
    last_purchase_date TIMESTAMP,
    last_sale_date TIMESTAMP,
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    deleted_at TIMESTAMP NULL,
    FOREIGN KEY (retailer_id) REFERENCES retailers(id) ON DELETE CASCADE
);

CREATE INDEX idx_items_retailer ON items(retailer_id) WHERE deleted_at IS NULL;
CREATE INDEX idx_items_category ON items(retailer_id, category);
CREATE INDEX idx_items_sku ON items(retailer_id, sku);
CREATE INDEX idx_items_barcode ON items(barcode) WHERE barcode IS NOT NULL;
CREATE INDEX idx_items_low_stock ON items(retailer_id)
    WHERE current_stock <= min_stock_threshold AND is_active = TRUE;

COMMENT ON TABLE items IS 'Product/inventory master table';
COMMENT ON COLUMN items.unit IS 'kg, liter, piece, packet, etc.';
COMMENT ON COLUMN items.purchase_price IS 'Cost price from supplier';
COMMENT ON COLUMN items.selling_price IS 'MRP to customer';

-- ============================================
-- 5. PURCHASE ORDERS TABLE
-- ============================================
CREATE TABLE purchase_orders (
    id BIGSERIAL PRIMARY KEY,
    retailer_id BIGINT NOT NULL,
    supplier_id BIGINT NOT NULL,
    order_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    status VARCHAR(20) DEFAULT 'pending',
    total_amount DECIMAL(12,2) NOT NULL,
    payment_status VARCHAR(20) DEFAULT 'pending',
    paid_amount DECIMAL(12,2) DEFAULT 0,
    notes TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    deleted_at TIMESTAMP NULL,
    FOREIGN KEY (retailer_id) REFERENCES retailers(id) ON DELETE CASCADE,
    FOREIGN KEY (supplier_id) REFERENCES suppliers(id) ON DELETE RESTRICT
);

CREATE INDEX idx_purchase_orders_retailer ON purchase_orders(retailer_id);
CREATE INDEX idx_purchase_orders_supplier ON purchase_orders(supplier_id);
CREATE INDEX idx_purchase_orders_status ON purchase_orders(retailer_id, status);
CREATE INDEX idx_purchase_orders_date ON purchase_orders(retailer_id, order_date DESC);

COMMENT ON TABLE purchase_orders IS 'Header table for purchases from suppliers';
COMMENT ON COLUMN purchase_orders.status IS 'pending, received, cancelled';
COMMENT ON COLUMN purchase_orders.payment_status IS 'pending, partial, paid';

-- ============================================
```

```
-- 6. PURCHASE ORDER ITEMS TABLE
-- ============================================
CREATE TABLE purchase_order_items (
    id BIGSERIAL PRIMARY KEY,
    purchase_order_id BIGINT NOT NULL,
    item_id BIGINT NOT NULL,
    quantity DECIMAL(10,2) NOT NULL,
    unit_price DECIMAL(10,2) NOT NULL,
    amount DECIMAL(12,2) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (purchase_order_id) REFERENCES purchase_orders(id) ON DELETE CASCADE,
    FOREIGN KEY (item_id) REFERENCES items(id) ON DELETE RESTRICT
);

CREATE INDEX idx_po_items_order ON purchase_order_items(purchase_order_id);
CREATE INDEX idx_po_items_item ON purchase_order_items(item_id);

COMMENT ON TABLE purchase_order_items IS 'Line items for each purchase order';
COMMENT ON COLUMN purchase_order_items.unit_price IS 'Purchase price snapshot at time of order';

-- ============================================
-- 7. SUPPLIER PAYMENTS TABLE
-- ============================================
CREATE TABLE supplier_payments (
    id BIGSERIAL PRIMARY KEY,
    retailer_id BIGINT NOT NULL,
    supplier_id BIGINT NOT NULL,
    purchase_order_id BIGINT,
    payment_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    amount DECIMAL(12,2) NOT NULL,
    payment_method VARCHAR(20),
    reference_number VARCHAR(100),
    notes TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (retailer_id) REFERENCES retailers(id) ON DELETE CASCADE,
    FOREIGN KEY (supplier_id) REFERENCES suppliers(id) ON DELETE RESTRICT,
    FOREIGN KEY (purchase_order_id) REFERENCES purchase_orders(id) ON DELETE SET NULL
);

CREATE INDEX idx_supplier_payments_retailer ON supplier_payments(retailer_id, payment_date DESC);
CREATE INDEX idx_supplier_payments_supplier ON supplier_payments(supplier_id);
CREATE INDEX idx_supplier_payments_po ON supplier_payments(purchase_order_id);

COMMENT ON TABLE supplier_payments IS 'Payment tracking for supplier dues';
COMMENT ON COLUMN supplier_payments.payment_method IS 'cash, cheque, upi, bank_transfer';

-- ============================================
-- 8. SALES TABLE
-- ============================================
CREATE TABLE sales (
    id BIGSERIAL PRIMARY KEY,
    retailer_id BIGINT NOT NULL,
    customer_id BIGINT,
    sale_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    total_amount DECIMAL(12,2) NOT NULL,
    discount DECIMAL(10,2) DEFAULT 0,
    final_amount DECIMAL(12,2) NOT NULL,
    payment_method VARCHAR(20),
    payment_status VARCHAR(20) DEFAULT 'paid',
    paid_amount DECIMAL(12,2) DEFAULT 0,
    notes TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    deleted_at TIMESTAMP NULL,
    FOREIGN KEY (retailer_id) REFERENCES retailers(id) ON DELETE CASCADE,
    FOREIGN KEY (customer_id) REFERENCES customers(id) ON DELETE SET NULL
);

CREATE INDEX idx_sales_retailer ON sales(retailer_id);
CREATE INDEX idx_sales_customer ON sales(customer_id);
CREATE INDEX idx_sales_date ON sales(retailer_id, sale_date DESC);
CREATE INDEX idx_sales_payment_status ON sales(retailer_id, payment_status);
```

```
COMMENT ON TABLE sales IS 'Customer bills/invoices';
COMMENT ON COLUMN sales.payment_method IS 'cash, card, upi, credit';
COMMENT ON COLUMN sales.payment_status IS 'paid, pending, partial';

-- ============================================
-- 9. SALE ITEMS TABLE
-- ============================================
CREATE TABLE sale_items (
    id BIGSERIAL PRIMARY KEY,
    sale_id BIGINT NOT NULL,
    item_id BIGINT NOT NULL,
    quantity DECIMAL(10,2) NOT NULL,
    unit_selling_price DECIMAL(10,2) NOT NULL,
    unit_cost_price DECIMAL(10,2) NOT NULL,
    amount DECIMAL(12,2) NOT NULL,
    profit DECIMAL(12,2) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (sale_id) REFERENCES sales(id) ON DELETE CASCADE,
    FOREIGN KEY (item_id) REFERENCES items(id) ON DELETE RESTRICT
);

CREATE INDEX idx_sale_items_sale ON sale_items(sale_id);
CREATE INDEX idx_sale_items_item ON sale_items(item_id);

COMMENT ON TABLE sale_items IS 'Line items for each sale';
COMMENT ON COLUMN sale_items.unit_selling_price IS 'Selling price snapshot at time of sale';
COMMENT ON COLUMN sale_items.unit_cost_price IS 'Purchase price snapshot for profit calculation';

-- ============================================
-- 10. INVENTORY TRANSACTIONS TABLE
-- ============================================
CREATE TABLE inventory_transactions (
    id BIGSERIAL PRIMARY KEY,
    retailer_id BIGINT NOT NULL,
    item_id BIGINT NOT NULL,
    transaction_type VARCHAR(20) NOT NULL,
    quantity DECIMAL(10,2) NOT NULL,
    reference_type VARCHAR(50),
    reference_id BIGINT,
    previous_stock DECIMAL(10,2),
    new_stock DECIMAL(10,2),
    notes TEXT,
    created_by VARCHAR(100),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (retailer_id) REFERENCES retailers(id) ON DELETE CASCADE,
    FOREIGN KEY (item_id) REFERENCES items(id) ON DELETE RESTRICT
);

CREATE INDEX idx_inventory_txn_retailer ON inventory_transactions(retailer_id, created_at DESC);
CREATE INDEX idx_inventory_txn_item ON inventory_transactions(item_id, created_at DESC);

COMMENT ON TABLE inventory_transactions IS 'Audit trail for all stock movements';
COMMENT ON COLUMN inventory_transactions.transaction_type IS 'purchase, sale, adjustment, return';
COMMENT ON COLUMN inventory_transactions.quantity IS 'Positive for stock-in, negative for stock-out';

-- ============================================
-- 11. RETURNS TABLE
-- ============================================
CREATE TABLE returns (
    id BIGSERIAL PRIMARY KEY,
    retailer_id BIGINT NOT NULL,
    sale_id BIGINT,
    customer_id BIGINT,
    return_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    total_amount DECIMAL(12,2) NOT NULL,
    refund_status VARCHAR(20) DEFAULT 'pending',
    reason TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (retailer_id) REFERENCES retailers(id) ON DELETE CASCADE,
    FOREIGN KEY (sale_id) REFERENCES sales(id) ON DELETE SET NULL,
    FOREIGN KEY (customer_id) REFERENCES customers(id) ON DELETE SET NULL
```

```
);

CREATE INDEX idx_returns_retailer ON returns(retailer_id, return_date DESC);

COMMENT ON TABLE returns IS 'Product returns from customers';
COMMENT ON COLUMN returns.refund_status IS 'pending, refunded';

-- ============================================
-- 12. RETURN ITEMS TABLE
-- ============================================
CREATE TABLE return_items (
    id BIGSERIAL PRIMARY KEY,
    return_id BIGINT NOT NULL,
    sale_item_id BIGINT NOT NULL,
    quantity DECIMAL(10,2) NOT NULL,
    amount DECIMAL(12,2) NOT NULL,
    FOREIGN KEY (return_id) REFERENCES returns(id) ON DELETE CASCADE,
    FOREIGN KEY (sale_item_id) REFERENCES sale_items(id) ON DELETE RESTRICT
);

CREATE INDEX idx_return_items_return ON return_items(return_id);

COMMENT ON TABLE return_items IS 'Line items for each return';

-- ============================================
-- 13. DAILY SUMMARY TABLE
-- ============================================
CREATE TABLE daily_summary (
    id BIGSERIAL PRIMARY KEY,
    retailer_id BIGINT NOT NULL,
    summary_date DATE NOT NULL,
    total_sales DECIMAL(12,2) DEFAULT 0,
    total_purchases DECIMAL(12,2) DEFAULT 0,
    total_profit DECIMAL(12,2) DEFAULT 0,
    cash_sales DECIMAL(12,2) DEFAULT 0,
    card_sales DECIMAL(12,2) DEFAULT 0,
    upi_sales DECIMAL(12,2) DEFAULT 0,
    credit_sales DECIMAL(12,2) DEFAULT 0,
    total_transactions INT DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (retailer_id) REFERENCES retailers(id) ON DELETE CASCADE,
    UNIQUE(retailer_id, summary_date)
);

CREATE INDEX idx_daily_summary_date ON daily_summary(retailer_id, summary_date DESC);

COMMENT ON TABLE daily_summary IS 'Pre-aggregated daily statistics for faster dashboard loading';

-- ============================================
-- VIEWS
-- ============================================

-- View 1: Low Stock Items
CREATE VIEW low_stock_items AS
SELECT
    i.*,
    r.business_name
FROM items i
JOIN retailers r ON i.retailer_id = r.id
WHERE i.current_stock <= i.min_stock_threshold
  AND i.is_active = TRUE
  AND i.deleted_at IS NULL;

COMMENT ON VIEW low_stock_items IS 'Items that need reordering';

-- View 2: Pending Supplier Payments
CREATE VIEW pending_supplier_payments AS
SELECT
    po.id,
    po.retailer_id,
    r.business_name,
```

```
    s.company_name as supplier_name,
    po.total_amount,
    po.paid_amount,
    (po.total_amount - po.paid_amount) as pending_amount,
    po.order_date
FROM purchase_orders po
JOIN retailers r ON po.retailer_id = r.id
JOIN suppliers s ON po.supplier_id = s.id
WHERE po.payment_status != 'paid'
  AND po.deleted_at IS NULL;

COMMENT ON VIEW pending_supplier_payments IS 'Outstanding amounts owed to suppliers';

-- View 3: Daily Sales Report
CREATE VIEW daily_sales_report AS
SELECT
    retailer_id,
    DATE(sale_date) as sale_day,
    COUNT(*) as total_bills,
    SUM(final_amount) as total_sales,
    SUM(final_amount - discount) as net_sales,
    SUM(CASE WHEN payment_method = 'cash' THEN final_amount ELSE 0 END) as cash_sales,
    SUM(CASE WHEN payment_method = 'card' THEN final_amount ELSE 0 END) as card_sales,
    SUM(CASE WHEN payment_method = 'upi' THEN final_amount ELSE 0 END) as upi_sales
FROM sales
WHERE deleted_at IS NULL
GROUP BY retailer_id, DATE(sale_date);

COMMENT ON VIEW daily_sales_report IS 'Aggregated daily sales breakdown by payment method';

-- ============================================
-- TRIGGERS FOR UPDATED_AT
-- ============================================

CREATE OR REPLACE FUNCTION update_updated_at_column()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = CURRENT_TIMESTAMP;
    RETURN NEW;
END;
$$ language 'plpgsql';

CREATE TRIGGER update_retailers_updated_at BEFORE UPDATE ON retailers
    FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();

CREATE TRIGGER update_suppliers_updated_at BEFORE UPDATE ON suppliers
    FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();

CREATE TRIGGER update_customers_updated_at BEFORE UPDATE ON customers
    FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();

CREATE TRIGGER update_items_updated_at BEFORE UPDATE ON items
    FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();

CREATE TRIGGER update_purchase_orders_updated_at BEFORE UPDATE ON purchase_orders
    FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();

CREATE TRIGGER update_sales_updated_at BEFORE UPDATE ON sales
    FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();

CREATE TRIGGER update_daily_summary_updated_at BEFORE UPDATE ON daily_summary
    FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();

-- ============================================
-- SAMPLE DATA INSERTION (Optional)
-- ============================================

-- Insert sample retailer
INSERT INTO retailers (business_name, owner_name, email, phone_number, address, gst_number,
business_type)
VALUES ('Sharma General Store', 'Rajesh Sharma', 'rajesh@example.com', '9876543210', 'Shop 12, MG Road,
Mumbai', '27AABCU9603R1ZM', 'kirana');
```

```
-- Note: Replace with actual retailer IDs in production
-- INSERT INTO suppliers (retailer_id, company_name, contact_person, phone)
-- VALUES (1, 'ABC Distributors', 'Ramesh Kumar', '9123456789');

-- ============================================
-- CONSTRAINTS & VALIDATIONS
-- ============================================

-- Stock should not be negative (optional - can be enforced at application level)
-- ALTER TABLE items ADD CONSTRAINT check_stock_positive CHECK (current_stock >= 0);

-- Amount validations
ALTER TABLE sales ADD CONSTRAINT check_sales_amount CHECK (final_amount >= 0);
ALTER TABLE purchase_orders ADD CONSTRAINT check_po_amount CHECK (total_amount >= 0);

-- Payment validations
ALTER TABLE sales ADD CONSTRAINT check_sales_payment CHECK (paid_amount <= final_amount);
ALTER TABLE purchase_orders ADD CONSTRAINT check_po_payment CHECK (paid_amount <= total_amount);

-- ============================================
-- PERFORMANCE OPTIMIZATION NOTES
-- ============================================

-- 1. All foreign keys have indexes
-- 2. Composite indexes on (retailer_id, frequently_queried_column)
-- 3. Partial indexes for active records (WHERE deleted_at IS NULL)
-- 4. Pre-aggregated daily_summary table
-- 5. Views for common complex queries

-- ============================================
-- SECURITY NOTES
-- ============================================

-- 1. All queries MUST filter by retailer_id
-- 2. Backend extracts retailer_id from JWT token
-- 3. Never expose raw retailer_id to frontend
-- 4. Use soft deletes (deleted_at) instead of hard deletes
-- 5. Validate stock availability before sales

-- ============================================
-- END OF SCHEMA
-- ============================================
```